

Git și GitHub, explicate de la zero

Control al versiunilor • repository • commit • branch • pull request



În prezentare folosim Snake pentru a înțelege conceptele Git. În Laboratorul 1, aplicația pe care o creați și o publicați pe GitHub se numește Lab01HelloAndroid.

De ce avem nevoie de Git?

Dezvoltăm aplicația Android Snake. În fiecare zi adăugăm funcționalități noi, dar uneori apar erori.



Cum păstrăm istoricul proiectului și revenim la o versiune funcțională?

Răspuns: folosind Git.

Ce este Git?

Git este un sistem de control al versiunilor.

Ne ajută să:

- ✓ urmărim modificările făcute în proiect
- ✓ păstrăm istoricul dezvoltării
- ✓ salvăm etape importante
- ✓ revenim la versiuni anterioare
- ✓ colaborăm cu alți programatori

Exemplu

Când adăugăm meniul jocului Snake, putem înregistra această etapă folosind Git.

Git funcționează local, chiar și fără internet.

De reținut: Git păstrează istoricul etapelor proiectului înregistrate prin commit-uri.

Care este diferența dintre Git și GitHub?



Git

Sistem de control al versiunilor

Instalat pe calculator

Creează commit-uri locale

Urmărește modificările



GitHub

Platformă online

Accesibil prin internet

Găzduiește repository-uri

Permite partajarea și colaborarea

Exemplu: Git înregistrează modificările jocului Snake, iar GitHub găzduiește repository-ul online.

Important: Git și GitHub nu sunt același lucru. Putem folosi Git fără GitHub.

Ce înseamnă LOCAL și REMOTE?



LOCAL

Proiectul de pe calculatorul nostru



`git push`



`git pull`



REMOTE

Repository-ul de pe GitHub

- Local = proiectul și istoricul de pe calculator.
- Remote = repository-ul aflat într-o altă locație, de exemplu pe GitHub.
- `git push` = trimitem commit-urile către GitHub.
- `git pull` = aducem și integrăm modificările din remote.

Important: modificările făcute pe calculator nu apar automat pe GitHub.

Ce este un Repository?

Un repository (repo) reprezintă proiectul împreună cu istoricul modificărilor înregistrate de Git.

```
Lab01HelloAndroid/  
├── app/  
├── gradle/  
├── build.gradle.kts  
├── settings.gradle.kts  
├── README.md  
└── .gitignore
```

Există:



Repository local – pe calculator



Repository remote – pe GitHub

Pentru laborator: repository-ul privat va avea numele nume-prenume-labSO.

Repository Public vs. Private

Pe GitHub, un repository poate fi Public sau Private.



PUBLIC

Oricine poate vedea repository-ul și codul sursă.

Exemplu: Un proiect personal pe care vrem să îl prezentăm altor programatori.



PRIVATE

Doar proprietarul și persoanele autorizate pot accesa repository-ul.

Exemplu: Un proiect de laborator, vizibil doar studentului și persoanelor cărora le acordă acces.

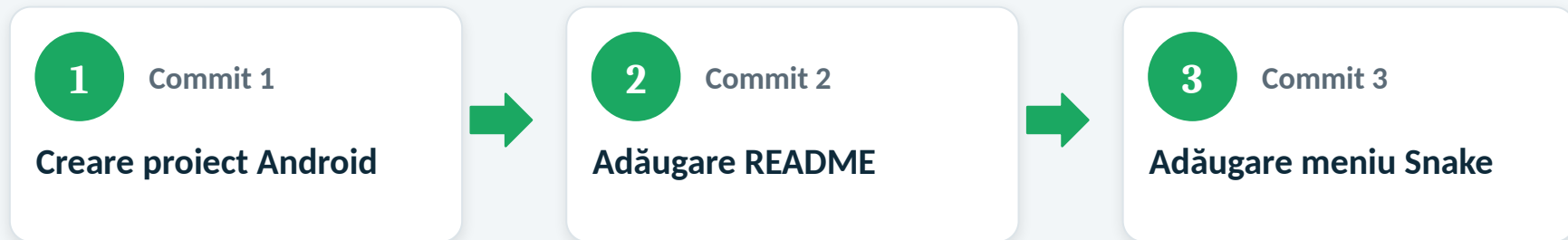
Important: Public nu înseamnă că oricine poate modifica codul; sunt necesare permisiuni.

În laborator, repository-ul este privat. Pentru ca profesorul să îl poată verifica, îi acordăm acces prin opțiunea Collaborators de pe GitHub.

De reținut: Public = vizibil tuturor. Private = acces restricționat.

Ce este un Commit?

Un commit este o versiune a proiectului înregistrată în istoricul Git.



Exemplu:

```
git commit -m "Lab 01 - creare proiect Android"
```

Un mesaj bun trebuie să descrie modificările realizate.

De reținut: un commit se creează local. Pentru a-l trimite pe GitHub folosim git push.

Ce este README.md?

README.md este un fișier care descrie proiectul. Conține, de obicei:

- numele proiectului;
- autorul;
- descrierea aplicației;
- instrucțiuni de instalare și rulare;
- alte informații utile.

Exemplu pentru laborator

```
# Lab 01 - Hello Android
```

```
Student: Popescu Andrei
```

```
Grupa: 123
```

```
GitHub: popescuandrei
```

```
Descriere: Prima aplicatie Android  
realizata folosind Java si XML.
```

De reținut: README este documentația de bază. Extensia .md înseamnă Markdown.

Ce este .gitignore?

Un fișier care indică ce fișiere sau directoare trebuie ignorate de Git.

Într-un proiect Android, unele fișiere sunt generate automat sau conțin configurări locale.

De ce îl folosim?

Pentru a evita includerea fișierelor temporare, a configurărilor locale și a cheilor private.

```
local.properties  
.gradle/  
.idea/  
*.iml  
build/  
app/build/  
*.jks  
*.keystore
```

Important: .gitignore nu elimină automat fișierele deja urmărite de Git.

Înainte de primul commit

Git trebuie instalat

Verificăm instalarea:

```
git --version
```

Configurăm identitatea Git

Numele și e-mailul identifică autorul commit-urilor. Înlocuiți exemplele cu datele voastre: adresa reală sau adresa GitHub noreply.

```
git config --global user.name "Nume Prenume"  
git config --global user.email "email-asociat-GitHub"
```

De reținut: configurarea globală se face, de regulă, o singură dată pe calculator.

Ce face git init?

Inițializează un repository Git local în directorul proiectului.

```
git init
```



Comanda creează directorul ascuns .git, care păstrează informațiile Git.

Important: git init nu creează un repository pe GitHub.

Ce face git status?

Ne arată starea fișierelor din proiect.

```
git status
```

Ne permite să vedem:

- fișiere noi;
- fișiere modificate;
- fișiere selectate pentru commit.

Exemplu

Modificăm MainActivity.java, apoi executăm git status pentru a vedea ce s-a schimbat.

```
MainActivity.java - modified
```

De reținut: git status doar afișează informații. Nu modifică fișierele.

Ce este Staging Area?

Zona în care selectăm modificările pentru următorul commit.



De reținut: `git add` nu creează un commit. Doar pregătește modificările.

git add . vs. git add fișier

Putem selecta toate modificările sau numai anumite fișiere.

1

Toate modificările neignorate (director curent + subdirectoare)

git add .

2

Un singur fișier

git add README.md

3

Două fișiere

**git add README.md
app/src/main/AndroidManifest.xml**

git add . selectează:



MainActivity.java



activity_main.xml



README.md

Sunt selectate toate cele trei fișiere modificate.

Important: git add README.md nu deselectează fișierele adăugate anterior în staging.

Cum creăm primul commit?

După ce am selectat modificările, le înregistrăm într-un commit.

```
git status
git add .
git status
git commit -m "Lab 01 - proiect Android initial"
```

- 1 Verificăm modificările.
- 2 Selectăm fișierele dorite.
- 3 Verificăm selecția.
- 4 Creăm commit-ul în repository-ul local.

Mesaje bune

- Lab 01 - creare proiect Android
- Lab 01 - adaugare README
- Lab 02 - implementare meniu

Mesaje de evitat

update, final2, ceva

De reținut: mesajul commit-ului descrie ce s-a modificat.

Cum creăm repository-ul pe GitHub?

După ce avem repository-ul local, creăm și un repository online.

- 1 Intrăm pe GitHub.
- 2 Selectăm New repository.
- 3 Introducem nume-prenume-labSO.
- 4 Alegem Private.
- 5 Nu adăugăm README, .gitignore sau licență.
- 6 Apăsăm Create repository.

GitHub ne oferă un link HTTPS:

```
https://github.com/USERNAME/nume-prenume-labSO.git
```

De reținut: acum avem două repository-uri, dar încă trebuie să le conectăm.

Cum conectăm repository-ul local la GitHub?

În terminalul Android Studio executăm:

```
git branch -M main
git remote add origin URL_REPOSITORY
git push -u origin main
```

URL_REPOSITORY se înlocuiește cu linkul HTTPS copiat de pe GitHub. De exemplu:

```
git remote add origin https://github.com/popescuandrei/popescu-andrei-labSO.git
```

LOCAL • Lab01HelloAndroid



GITHUB • nume-prenume-labSO

<code>git branch -M main</code>	Redenumeste branch-ul curent în main
<code>git remote add origin URL</code>	Configurează adresa repository-ului remote
<code>git push -u origin main</code>	Trimite branch-ul pe GitHub și stabilește legătura implicită

Important: nu executăm `git remote add origin` la fiecare laborator. Este o configurare inițială.

Ce sunt Push, Pull și Clone?

git push

Trimite commit-urile locale pe GitHub

git pull

Aduce și integrează modificările remote

git clone URL

Creează o copie locală a unui repository existent

Exemplu: am făcut un commit pe calculator și vrem să apară pe GitHub → **git push**

De reținut: git clone copiază un repository existent. git pull actualizează unul deja existent.

Al doilea commit: adăugăm README

Creăm fișierul README.md în directorul principal al proiectului. Apoi executăm:

```
git status
git add README.md
git status
git commit -m "Lab 01 - adaugare README"
git push
```

După push, verificăm că fișierul README și commit-ul apar în repository.

Ce se întâmplă?

- Verificăm modificările.
- Selectăm fișierul README.
- Verificăm selecția.
- Înregistrăm commit-ul local.
- Tritem commit-ul pe GitHub.

De reținut: push-ul trimite commit-ul local pe GitHub.

BRANCH-URI ȘI LUCRUL ÎN ECHIPĂ

Aceste concepte sunt prezentate pentru a înțelege colaborarea într-o echipă de programatori.

În Laboratorul 1 lucrăm doar pe branch-ul main.

Ce sunt Branch și Main?

Un branch este o linie separată de dezvoltare. Ne permite să lucrăm la funcționalități noi fără să modificăm imediat versiunea principală.

main

branch-ul principal

feature/meniu

branch pentru meniul jocului

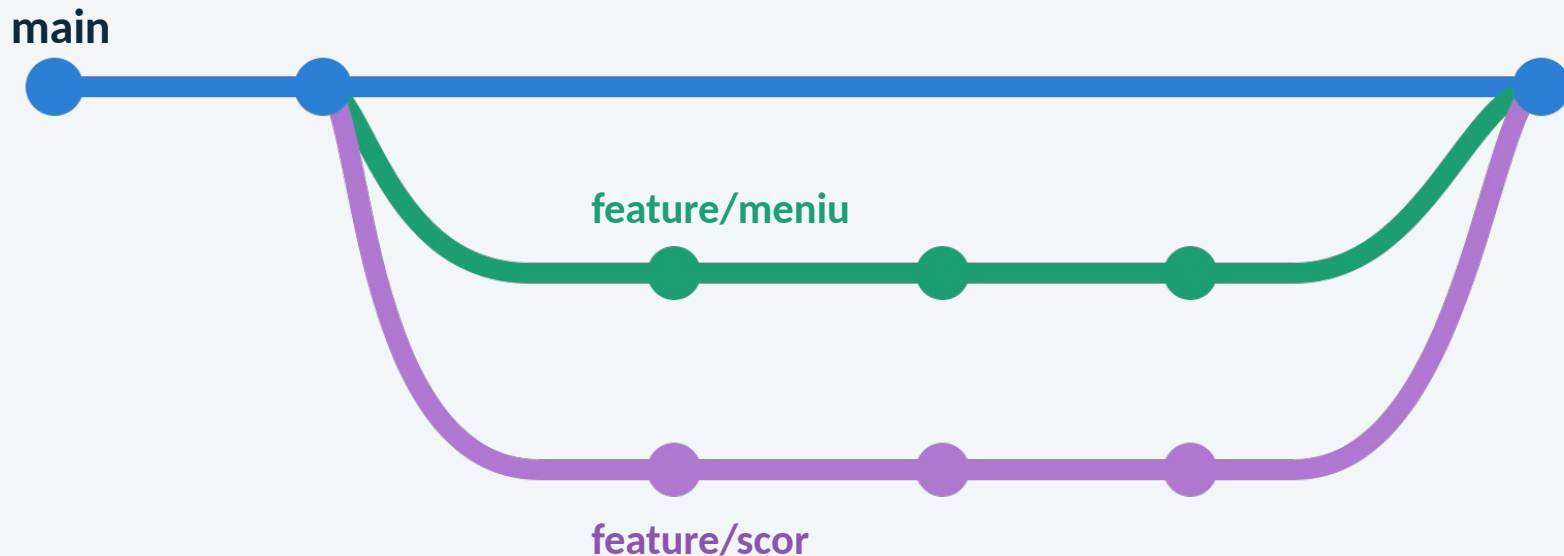
feature/scor

branch pentru sistemul de scor

Exemplu: Andrei dezvoltă meniul, iar Maria dezvoltă scorul, fiecare pe branch-ul propriu.

De reținut: main = versiunea principală a proiectului.

Cum arată un Branch?



Schemă simplificată: dezvoltarea separată a funcționalităților și integrarea lor în main.

Cum lucrăm pe un Branch?

Exemplu: dezvoltăm meniul jocului Snake.

```
git switch -c feature/meniu

# Modificăm și testăm codul aplicației

git status
git add .
git commit -m "Adaugare meniu"
git push -u origin feature/meniu
```

Ce face fiecare pas?

- switch -c: creează și activează branch-ul
- Modificăm și testăm codul: branch-ul nu implementează singur funcționalitatea
- status: verificăm ce fișiere sunt modificate și care sunt deja selectate pentru commit
- add: selectează modificările
- commit: înregistrează local
- push: trimite branch-ul pe GitHub

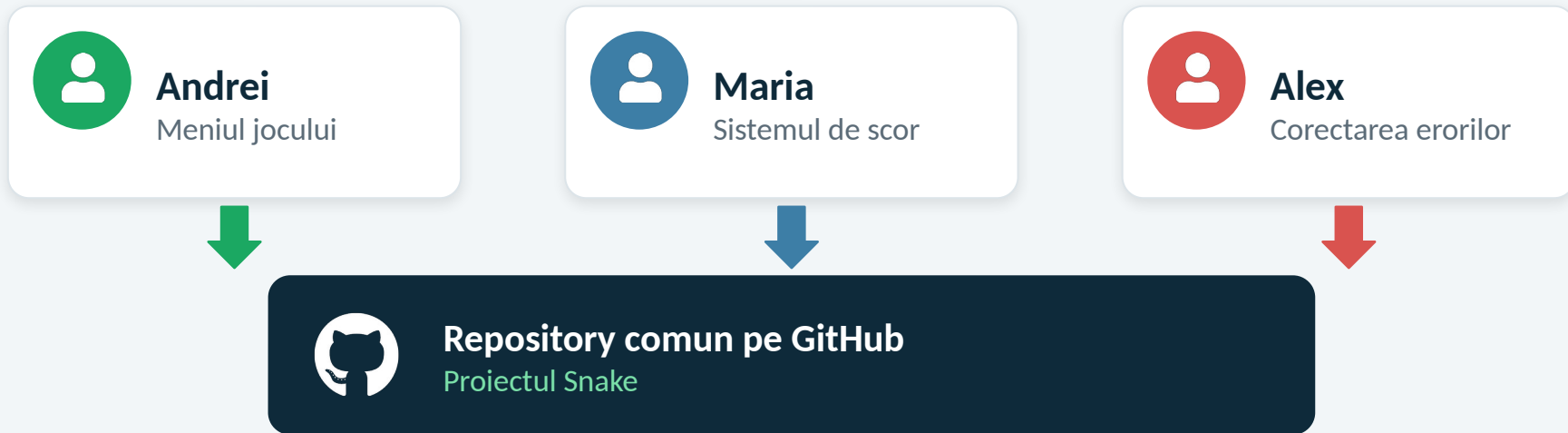
Important: pentru laboratoarele actuale lucrăm pe main. Branch-urile sunt prezentate pentru a înțelege lucrul în echipă.

PARTEA III

Cum se lucrează cu Git într-o companie

Trei programatori, un singur proiect

O companie IT dezvoltă jocul Android Snake. La proiect lucrează trei programatori:



Fiecare lucrează pe un branch separat, iar modificările sunt ulterior integrate.

De ce nu lucrează toți direct pe main? Pentru a verifica funcționalitățile separat, înainte de integrare.

Ce este un Pull Request?

O cerere de integrare a modificărilor dintr-un branch în alt branch.

Andrei a terminat meniul Snake:

- 1 Creează un commit.
- 2 Trimite branch-ul pe GitHub.
- 3 Creează un Pull Request către main.
- 4 Un coleg verifică modificările.
- 5 După aprobare, modificările pot fi integrate.

Pull Request #12 **Open**

Adăugare meniu principal

`feature/meniu` → `main`

Code review

Un coleg verifică funcționalitatea și poate solicita corecturi.

De reținut: git push trimite codul pe GitHub, dar nu îl integrează automat în main.

Cum creăm un Pull Request?

Pull Request-ul se creează pe GitHub, din interfața web. Nu avem nevoie de comenzi în terminal.

Pe GitHub:

- 1 Deschidem repository-ul.
- 2 Apăsăm Compare & pull request (sau Pull requests → New pull request).
- 3 Alegem main ca destinație și feature/meniu ca sursă.
- 4 Scriem titlul și descrierea modificărilor.
- 5 Apăsăm Create pull request.

feature/meniu had recent pushes

Compare & pull request

base: **main**
compare: **feature/meniu**
Adăugare meniu Snake

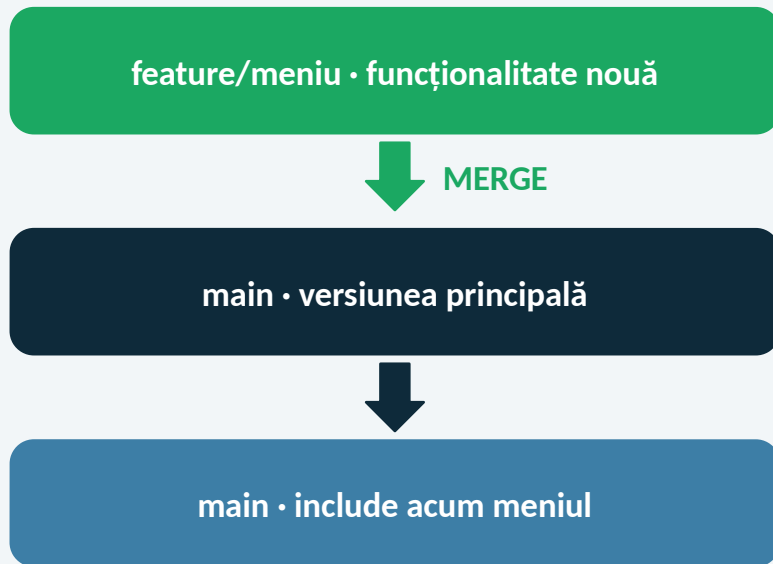
Apoi: un coleg verifică codul; după aprobare, cineva cu permisiuni apasă Merge pull request.

Ce este Merge?

Operația prin care integrăm modificările dintr-un branch în altul.

Andrei a terminat meniul Snake:

- Codul este pe branch-ul feature/meniu.
- Colegii verifică modificările.
- Modificările sunt integrate în main.
- Versiunea principală include acum meniul.



Important: dacă două modificări sunt incompatibile, poate apărea un merge conflict, rezolvat înainte de integrare.